

Style

Today's agenda:

- Code style (R for Data Science Chapter 4)
- Data style (R for Data Science Chapter 5)
- Pseudocode

Code style

- Makes code easier to read
- Makes code more understandable (to others and future-you)
- Makes it easier to get help
- Makes you look impressive

Load packages and data

```
library(tidyverse)
```

```
library(styler) # style package
```

```
library(lintr) # style package
```

```
avo <- read_rds("https://tinyurl.com/avonetdata")
```

Run these now, installing packages if needed.

File Names

- Use “snake case”: words_separated_by_an_underscore
- Name should describe file contents
- Lowercase only
- NO special characters or spaces. Only numbers, letters, . and _

Examples:

`avonet_bird_trait_data.csv`

`BirdData.csv`

`Bird data?.csv`

Which is best?

File Names

File names:

- Use similar structure for similar files
- If order matters:
 - Put numbers in front of file names (use 01 rather than 1 so they sort correctly)
 - If dated, use syntax yyyyymmdd so they order correctly
- Don't put "final" in the name
 - Git will track changes for you (no need for multiple versions)
 - If you want multiple versions, give version number or date in the name

Examples:

01_download_data.R

02_clean_data.R

avonet_data_clean_20251117.rds

Object Names

Object names:

- Use “snake case”: `words_separated_by_an_underscore`
 - Note: our book often uses “`words.separated`” because it uses an older naming convention
- No special characters
- Lowercase only
- Variable names should be nouns

What would be a good name for an object containing:

Avonet data subsetting to only Hawks?

Avonet data subsetting to only forest birds?

Object Names

Function names:

- Should be verbs explaining what the function does
- Avoid using existing function names in R

Suggest names for functions that:

Convert the avonet data to family-level means

Load data from Github

Object Names

- Better to have long, informative names
- If you have a lot of related objects, you can give them a common prefix

Examples:

```
Avonet_family_level_Psittacidae
```

```
Avonet_family_level_Paridae
```

This is clear, and autocomplete makes it fast

Object names

Which of these names is good? Why?

```
parrot_data <- avo |> filter(Family1 == "Psittacidae")
```

```
parrot-data <- avo |> filter(Family1 == "Psittacidae")
```

```
parrot data <- avo |> filter(Family1 == "Psittacidae")
```

Spaces

- Aid in readability
- Can break code if put in the wrong place
- Put spaces between objects and mathematical/logical operators (except for ^)

Example:

```
a <- 1 + 2
```

```
b <- 2^3
```

Spaces

- Spaces after commas
- No spaces next to parentheses in function calls
- No spaces next to \$ or @

Example:

```
mean(x = avo$Mass, na.rm = TRUE)
```

Spaces

Its ok to add extra spaces to improve alignment or readability

Example:

```
avo |>
  group_by(Family1) |>
  summarise(mean_mass      = mean(Mass),
             sd_mass       = sd(Mass),
             mean_wing_length = mean(Wing.Length),
             sd_wing_length  = sd(Wing.Length))
```

Pipes and arguments

- Pipes should go at the end of a line
- Named arguments should each start on a new line

Example:

```
avo |>
  group_by(Family1) |>
  summarise(mean_mass = mean(Mass),
            sd_mass = sd(Mass),
            mean_wing_length = mean(Wing.Length),
            sd_wing_length = sd(Wing.Length))
```

Pipes and arguments

- The “->” is generally frowned upon, even though it makes more sense when reading code
- Putting it at the start makes it clear you're making a new variable

Example:

```
avo |>
  group_by(Family1) |>
  summarise(mean_mass = mean(Mass),
            sd_mass = sd(Mass),
            mean_wing_length = mean(Wing.Length),
            sd_wing_length = sd(Wing.Length)) -> family_level_summary
```

Breaking up code bits

- Putting an empty line between functions can make them more readable
- You can also break up section of code using:
 - ctrl(cmd)+shift+R to add a section divider
 - A bunch of comments together to make a section divider

```
# code chunks -----  
  
#####
```

Try out ctrl(cmd)+shift+R now to create a section label

Having R style your code for you

- The lintr package checks your code and gives you style feedback
- The styler package automatically restyles your code for you
- I prefer “lintr”, since there are some style conventions I ignore

Try it out:

```
lint("an_R_file.R") # but obviously with your own filename
```


Data style

Wide vs Narrow/Long

	Wide	Narrow/long
Structure	Many columns. Each row contains all the information about a subject.	Few columns. One measurement per row. Often have ID columns to link multiple measurements on a single individual
Pro	Easier to enter, clear how data relate to each other	Can easily add new information
Con	Can be larger when you have many unmeasured variables, harder to add new data types	Less intuitive, may not play well with plotting functions, requires ID fields

Data style

Wide vs Narrow/Long

	Species1	Mass	Wing.Length
1	Accipiter albogularis	248.8	235.2
2	Accipiter badius	131.2	186.7
3	Accipiter bicolor	287.5	229.6

	Species1	trait	trait_value
1	Accipiter albogularis	Mass	248.8
2	Accipiter albogularis	Wing.Length	235.2
3	Accipiter badius	Mass	131.2
4	Accipiter badius	Wing.Length	186.7
5	Accipiter bicolor	Mass	287.5
6	Accipiter bicolor	Wing.Length	229.6

Data style

Tidy data:

- Every row is an observation
- Every column is a variable
- Each cell has a single value

This structure is intuitive and plays well with tidyverse

- There may be multiple different “tidy” options
- Getting your data into tidy format may require some work

Converting between styles

```
pivot_longer() # lengthens data
```

```
pivot_wider() # widens data
```

Let's grab some data to work with:

```
avo_wide <-
```

```
  avo |>
```

```
    select(Species1, Mass, Wing.Length, Beak.Width,  
           Beak.Length_Culmen)
```

Inspect the `avo_wide` dataset. Is this tidy?

Converting between styles

Convert to long format:

```
avo_long <-  
  avo_wide |>  
  pivot_longer(cols = Mass:Beak.Length_Culmen,  
               names_to = "trait",  
               values_to = "trait_value")
```

How many lines are there per species?

How could we convert back to the original format?

Converting between styles

Convert to wide format:

```
avo_wide2 <-  
  avo_long |>  
  pivot_wider(id_cols = Species1,  
              names_from = trait,  
              values_from = trait_value)
```

Does `avo_wide2` contain the same thing as `avo_wide`?

Pseudocode

- A powerful coding technique that works with **any** code language
- Informal, high-level description of an other algorithm.
- Uses the structural conventions of programming language, but is
- Intended for human reading rather than machine reading.

Very helpful when your code gets more complicated!

Pseudocode

Essentially, you write out each code step of your workflow before writing the code

- Forces you to think about the whole algorithm/approach

Pseudocode can be added as comments to a new file

- Then you just have to figure out how to address each step
- Breaks down workflows into Google-able chunks

Pseudocode

Say we wanted to identify largest, mass-corrected, difference in beak lengths

Our pseudocode might be:

```
# Calculate difference in beak lengths (culmen vs nares)
```

```
# Divide difference by body mass
```

```
# Identify largest mass-corrected difference in beak lengths
```

Pseudocode

Then we fill in the code:

```
avo |>
```

```
# Calculate difference in beak lengths (culmen vs nares)
```

```
  mutate(delta_beak_length = Beak.Length_Culmen - Beak.Length_Nares) |>
```

```
# Divide difference by body mass
```

```
  mutate(mass_corrected_delta_beak_length = delta_beak_length / Mass) |>
```

```
# Identify largest mass-corrected difference in beak lengths
```

```
  slice_max(order_by = mass_corrected_delta_beak_length,  
            n = 1) |>
```

```
  select(Species1, mass_corrected_delta_beak_length)
```

Comments

We've been using comments for a while now

- You can use them to “turn off” bits of code temporarily
- Can also make useful notes on your code
- One of the most important things to remember to note is “why”
 - Why did you make a particular decision in your code?
 - Why did you use one package and not another?
 - Why did you exclude some data?

Comments

Well-commented code is super helpful

- Makes it easier for others to use it
- Makes it easier for you to find parts to change it
- Makes it easier to write a “Methods” section
 - I often start by transcribing my comments and then cleaning up the text a bit

Combined use of comments and pseudocode

Here's what I often do:

- 1) Start a new R script
- 2) Add comments documenting what the file is for
- 3) Add in pseudocode of the intended workflow
- 4) Write out your full workflow
 - a) Add comments where needed to clarify decisions you made
- 5) Run the workflow
- 6) Use the comments to write the methods section of the paper

Before next class:

- Read “R for Data Science” chapter 1:
 - <https://r4ds.hadley.nz/data-visualize.html>
- May be a quiz